

Cheatsheet — Neovim & tmux · config de accel

NEOVIM

Leader = **Espacio**. La mayoría de comandos empiezan con Espacio + una letra que recuerda la acción. **Tip:** presiona **Espacio** y espera un momento → **which-key** te muestra un menú con todas las opciones disponibles.

e · Explorer (árbol de archivos)	f · Find (búsqueda difusa)	s · Split (ventanas)
_ · Buffers (alternar)	l · LazyGit (todo lo de git)	x · Trouble (errores)
m · forMat (prettier/stylua)	w · Workspace (sesión)	g · Goto · r/c · rename/code

General & Movimiento

kj

/

jk

Salir a modo normal estando en insert
Más rápido que estirar la mano al Esc

u

nh

Quitar el resaltado de la última búsqueda
nh = No Highlight. Tras buscar con / queda todo pintado; esto lo limpia

gb

Volver atrás en el historial de saltos
Regresa al lugar donde estabas antes de un salto (gd, búsqueda, etc.)

u

+

/

u

-

Incrementar / decrementar el número bajo el cursor

Portapapelessolo lo que copias con y va al portapapeles del sistema

y

yy

Copiar (yank) → portapapeles de macOS
Lo pegas con Cmd+V en cualquier otra app

p

Pegar el portapapeles
Sirve para pegar tanto tus yanks como lo que copiaste fuera con Cmd+C

d

c

x

Borrar / cambiar SIN tocar el portapapeles
Van al "black hole": borras cosas sin perder lo que tenías copiado

u

d

Cortar Sí enviando al portapapeles
⚠ En archivos con LSP esta tecla muestra el diagnóstico de la línea (gana el LSP)

p

(visual)

Pegar encima de una selección sin perder lo copiado

Ventanas / Splitss = Split

u

sv

Split vertical (dos paneles lado a lado)

u

sh

Split horizontal (uno arriba, otro abajo)

u

se

Igualar el tamaño de los splits

u

sx

Cerrar el split actual

u

sm

Maximizar / restaurar el split (foco temporal)

u

sr

Modo resize incremental: luego tap

h/l

 (ancho) o

j/k

 (alto), repetible; otra tecla sale
Ideal para agrandar nvim-tree: `__sr` y tap `lll`

^h

^j

^k

^l

Moverte entre splits (y entre panes de tmux)
Mismas teclas funcionan en vim y en tmux gracias a vim-tmux-navigator

split vs pane

Split de nvim = dividir el EDITOR (dos vistas de código en el mismo nvim, comparten yank/LSP/saltos).
Pane de tmux = dividir el TERMINAL (editor + shell/tests, procesos independientes)
Regla: código-vs-código → split de nvim · editor-vs-shell → pane de tmux

Buffers

un buffer = un archivo abierto en memoria



Alternar entre el buffer actual y el anterior

Ya NO hay barra de tabs arriba (se quitó: se acumulaban). El archivo actual se ve abajo, en lualine



Buffers sin guardar (:ls +) · `:ls` = todos

El + en la lista marca los modificados. Salta con :b N

pestañas / terminal

Ya **no las maneja nvim**: las "pestañas" son **ventanas de tmux** (prefix + c/n/p) y para una shell usas un **pane de tmux** (Ctrl+j al de abajo)

Los tabpages nativos de nvim siguen existiendo con :tabnew/tabn si algún día los quieres, pero sin atajos de leader

Explorador de archivos (nvim-tree)

e = Explorer



Abrir / cerrar el árbol de archivos



Ubicar el archivo actual dentro del árbol



Enfocar el árbol (saltar el cursor ahí)



Colapsar todo · **Refrescar**



Duplicar el archivo seleccionado

Dentro del árbol

teclas cuando el cursor está en el explorador



Crear archivo o carpeta

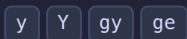
Termina el nombre en / para crear carpeta (ej. components/)



Renombrar · **Borrar** (va a la Papelera)



Cortar · **Copiar** · **Pegar** archivos



Copiar al portapapeles:

y = nombre (index.js) · **Y** = ruta relativa (src/index.js) · **gy** = ruta absoluta (/Users/.../src/index.js) · **ge** = nombre sin extensión (index)

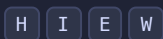


Abrir · en **split vertical** · en **split horizontal** · en **pestaña nueva**

⚠ ^t abre un tabpage que ya NO se ve (sin barra arriba): sales con `:tabc`. Usa ^v / ^x



Previsualizar sin salir del árbol · ir a la **carpeta padre** · **subir** un nivel de raíz



Ver ocultos (dotfiles) · ver **git-ignored** · **Expandir todo** · **Colapsar todo**



Ayuda: lista TODOS los atajos del árbol

Búsqueda (Telescope)

f = Find

 ff	Buscar archivos por nombre en el proyecto Escribe parte del nombre; es difuso (no necesita ser exacto)
 fr	Archivos recientes (los que abriste últimamente)
 fb	Buffers abiertos (lista difusa de lo que tienes abierto)
 fs	Buscar texto dentro de todos los archivos (live grep) Busca una palabra/frase en TODO el proyecto en tiempo real
 fc	Buscar la palabra bajo el cursor en el proyecto
 fa	Búsqueda amplificada solo en los archivos abiertos
 ^j ^k ↵	Dentro del buscador: bajar / subir (navegar) · abrir el resaltado ^j/^k navegan (NO uses Tab para eso)
Tab ^q	Marcar varios (multi-selección, sale un +) →  q los manda al quickfix Para refactors: marca los usos y recórrelos con <code>_xq / :cnext</code>

🔗 LSP: ¿cuál uso? (definición vs declaración vs referencias...)

Definición (gd): te lleva a donde algo está **escrito/implementado de verdad**. Ej: sobre `miFuncion()` → salta al cuerpo de esa función. **Es el que más usarás.**

Declaración (gD): dónde se **declara** (se anuncia que existe) sin necesariamente implementarlo. En JS/TS suele coincidir con la definición; se nota más en C/C++ (te lleva al `.h`).

Referencias (gr): **todos los lugares donde se usa** ese símbolo en el proyecto. Ideal para "¿qué se rompe si cambio esto?".

Implementaciones (gi): para **interfaces / clases abstractas**: dónde se implementan concretamente. Ej: una interfaz `Repository` → sus clases que la cumplen.

Definición de tipo (gt): te lleva a la **definición del tipo** de una variable. Ej: sobre `const u: User` → salta a `interface User`.

Hover (K): muestra **documentación y tipo** en un popup, sin moverte del lugar.

Servidor TS: tsgo (TypeScript 7 nativo en Go, preview — más rápido, soporta monorepos). Para volver al clásico ts_ls: `use_tsgo = false` en `lsrconfig.lua`. Node (`fs`, `path`...) autocompleta si hay `@types/node` en la carpeta padre.

LSP: navegación por el código

g = Goto

gd	Ir a la definición (dónde se implementa)
gD	Ir a la declaración
gr	Ver referencias (todos los usos) — dispara directo Se quitaron los defaults <code>grr/grn/gra/gri/grt</code> de nvim 0.11 que hacían esperar el menú
gi	Ver implementaciones (de una interfaz)
gt	Ir a la definición del tipo
K	Documentación / tipo en popup (hover, con borde)
^s	Signature help: los parámetros de la función Útil al escribir la llamada; funciona en normal e insert

LSP: acciones y diagnósticos

c=code · r=rename · d=diagnostic

 ca	Code actions: arreglos que ofrece el editor Auto-importar, corregir error, refactor rápido, etc.
 rn	Renombrar símbolo en todo el proyecto (smart rename) Cambia la variable/función en todos sus usos a la vez
 rs	Reiniciar el LSP (si se cuelga o deja de sugerir)
 d	Ver el error/warning de la línea (popup)
 D	Lista de todos los diagnósticos del archivo
[d /]d	Saltar al diagnóstico anterior / siguiente
 ih	Inlay hints: mostrar/ocultar tipos inline Anotaciones grises de tipos y nombres de parámetro (i=inlay h=hints). Vienen encendidos por defecto

Git

todo el flujo va por LazyGit

 l	Abrir LazyGit: interfaz completa de git (stage, commit, push, ramas, diff, blame...) Teclas de lazygit: ver docs/lazygit-cheatsheet.md
<i>marcas</i>	Las marcas del margen (verde/azul/rojo) las pinta git. signs: líneas añadidas / modificadas / borradas vs. el último commit Solo informativas — los atajos de hunks (_h*) se quitaron; todo se hace en LazyGit

Trouble — panel de errores

x

 xx	Abrir/cerrar el panel Trouble
 xd / xw	Diagnósticos del documento / de todo el workspace
 xq / xl	Lista quickfix / loclist en formato bonito
 xt	TODOs del proyecto (TODO, FIXME, HACK...)

Formato

m = forMat

 mp	Formatear el archivo o la selección Usa prettier (JS/TS/JSON/YAML/MD), stylua (Lua) o google-java-format (Java)
<i>automático</i>	Se formatea solo al guardar . El lint lo hace en vivo el LSP de eslint (solo en proyectos con config de eslint)

Sesiones de trabajo (auto-session)

w = Workspace

 ws	Guardar la sesión de esta carpeta (archivos abiertos, layout)
 wr	Restaurar la sesión de esta carpeta Vuelves a un proyecto y recuperas todo como lo dejaste

Comentarios & Surround

gc = go comment · s = surround

`gcc`

Comentar / descomentar la línea actual (toggle)

`gc` + `mov`

Comentar por movimiento

`gc3j` = 3 líneas · `gcip` = párrafo · `gcG` = hasta el final

`gc` (*visual*) /  /

Comentar la selección

En JSX/HTML el estilo de comentario se ajusta solo

`ys` {`mov`}{`c`}

Rodear con comillas/paréntesis/tag

ej. `ysiw` → rodea la palabra con comillas: `"palabra"`

`ds` {`c`} `cs` {`a`}{`b`}

Borrar envoltorio (`ds`) · **Cambiar** envoltorio (`cs`"" pasa de " a ')

Autocompletado (en modo insert)

nvim-cmp

`^Space`

Mostrar sugerencias manualmente

`^j` / `^k`

Bajar / subir en la lista de sugerencias

`↵` / `→`

Confirmar la sugerencia seleccionada

`^b` / `^f` `^e`

Scroll de la documentación · **cerrar** el menú

TMUX

Prefix = `Ctrl+a`. Casi todo empieza así: presionas `Ctrl+a`, **sueeltas**, y luego la tecla. Abajo lo escribo como `^a` `tecla`.

Modelo mental — tmux tiene 3 niveles:

Sesión = un proyecto completo (ej. "kambista"). Sobrevive aunque cierres la terminal.

Window (ventana) = una pestaña dentro de la sesión (ej. editor / servidor / git).

Pane (panel) = una división visual dentro de una ventana.

Tus sesiones se **auto-guardan cada 15 min** y se **restauran solas** al reiniciar (resurrect + continuum).

Sesiones

lo que sobrevive al cerrar la terminal

<code>tmux new -s api</code>	Crear sesión con nombre (desde la terminal) Hazlo siempre con nombre para reconocerla luego
<code>tmux ls</code>	Listar sesiones
<code>tmux attach -t api</code>	Entrar a una sesión existente
<code>^a</code> <code>d</code>	Detach : salir dejando todo corriendo por detrás La tecla estrella de tmux. Cierras la terminal y nada se pierde
<code>^a</code> <code>s</code>	Árbol de sesiones (navega ↑↓, Enter entra) Úsalo para saltar entre sesiones SIN el error de "nesting"
<code>^a</code> <code>\$</code>	Renombrar la sesión
<code>^a</code> <code>(</code> / <code>)</code>	Sesión anterior / siguiente
<code>^a</code> <code>L</code>	Última sesión donde estuviste (toggle)
<code>tmux kill-server</code>	Matar todo y empezar de cero (desde la terminal)

Windows (pestañas)

<code>^a</code> <code>c</code>	Crear window (<i>c = create</i>)
<code>^a</code> <code>r</code>	Renombrar la window (pre-rellena el nombre actual) (<i>, también sirve</i>)
<code>^a</code> <code>1</code> ... <code>9</code>	Ir a la window N (<i>las tuyas empiezan en 1</i>)
<code>^a</code> <code>h</code> / <code>l</code>	Anterior / siguiente window (minúscula = window). Repetible <i>n / p</i> por defecto también funcionan
<code>^a</code> <code>w</code>	Árbol de windows de todas las sesiones
<code>^a</code> <code>&</code>	Cerrar la window (pide confirmación)
<code>^a</code> <code>f</code>	Buscar window por texto

Panes (paneles)

donde vives el 80% del tiempo

^a v	Split vertical (lado a lado)
^a -	Split horizontal (arriba / abajo)
^a x	Cerrar el pane (sin confirmación)
^h ^j ^k ^l	Moverte entre panes — SIN prefix Las mismas teclas saltan también a los splits de Neovim
^a H/L j/k	Redimensionar el pane: H/L (izq/der, MAYÚSCULA) · j/k (abajo/arriba). Repetible Regla: MAYÚSCULA = resize pane · minúscula h/l = cambiar window
^a m	Zoom : el pane a pantalla completa; repite para volver Úsalo mucho para enfocarte en un pane
^a q	Mostrar números de pane → presiona el número para saltar
^a Space	Rotar layouts predefinidos
^a { / }	Intercambiar pane con el anterior / siguiente
^a !	Convertir el pane en una window propia

Copy mode (scroll y copiar)

estilo vim

^a C	Entrar a copy mode (<i>[también sirve]</i>) · o solo scrollea con el mouse
h j k l / gg G	Moverte como en vim (buscar con /, inicio gg, fin G)
v → mover → y	Seleccionar y copiar al portapapeles de macOS Con y te quedas en copy mode; luego pegas con Cmd+V donde sea
⬮ arrastrar	Seleccionar con mouse → al soltar, copia al portapapeles y sale
q	Salir de copy mode

Config, plugins y ayuda

^a R	Recargar la config (MAYÚSCULA; r pasó a renombrar window)
^a I	Instalar plugins nuevos (tpm) (<i>I mayúscula</i>)
^a U	Actualizar plugins
^a ?	Ver TODOS los atajos activos
^a :	Línea de comandos de tmux
^a ^s / ^r	Guardar / restaurar sesiones manualmente (resurrect)

Notas útiles de tmux: el **mouse está activado** (click para elegir pane, arrastra bordes para redimensionar, scroll, click en pestañas). Las windows y panes **empiezan en 1** y se **renumeran** al cerrar. Para mandar un **Ctrl+a** literal a un programa, presiónalo **dos veces**.



TERMINAL (zsh + WezTerm)

Tu shell es **zsh** en **WezTerm**. Estas teclas ya están configuradas para sentirse como macOS.

💡 **La sugerencia gris (autosuggestions)** aparece sola según tu historial mientras escribes — NO es lo mismo que buscar con ↑.

→ (**flecha derecha**) o **End**: aceptar TODA la sugerencia gris.

Option + →: aceptar solo UNA palabra de la sugerencia.

↑ / ↓: otra cosa — busca en el historial lo que empieza con lo que ya escribiste.

Edición de línea (estilo macOS)

⌘ ← / ⌘ →

Saltar por palabra (izquierda / derecha)

⌘ ← / ⌘ →

Inicio / fin de la línea

⌘ ⌫

Borrar hasta el inicio de la línea (como en macOS)

⌘ ⌫

Borrar la palabra anterior

^k

Borrar del cursor **hasta el final**

^a / ^e

Inicio / fin (equivalente sin mouse)

Autosugerencias & historial

→ / End

Aceptar la sugerencia gris completa

⌘ →

Aceptar **una palabra** de la sugerencia

↑ / ↓

Buscar en historial por el prefijo que escribiste

^r

Búsqueda interactiva en todo el historial

Carpetas & archivos (zoxide + eza)

cd (= z)

zoxide: salta a carpetas frecuentes → `z kadmin`, `z uncc files`

z -

Volver a la **carpeta anterior**

zi

Elegir carpeta en modo **interactivo** (fzf)

ls (= eza)

Listar archivos con íconos

Control & limpieza

^l

Limpiar la pantalla (= clear)

^c

Cancelar el comando actual

^d

Cerrar la shell (EOF)

Tab

Autocompletar comando / ruta

Tus aliases & scripts

vim

abre **nvim**

eslint-init

configura ESLint en el proyecto actual

cheatsheet

regenera este PDF

denos

deno run con permisos de env/net